

Boris' Lockpicks

Sikkerhetsrevisjon 2024



HKSecurity

BL Sikkerhetsrevisjon: 2024-11-21 FPI PEN-T ver. 1.0 kandidatnr. 50200

Boris' Lockpicks – sikkerhetsrevisjon 2024 – Client Confidential

Contents

Executive Summary	4
Konfidensialitetsavtale	5
Ansvarsfraskrivelse	5
Omfang	5
Versjonslogg	6
Forkortelser	6
Gjennomførelse	7
Deltagere	7
Verktøy	7
Tidslinje	7
Testtype	7
Oversikt over sårbarheter	8
Sårbarhetsvurdering	8
Risikomatrise	8
Sårbarhetstabell	9
Detaljert beskrivelse av funn	10
Sårbarhet 1 – SQLI	10
Sårbarhet 2 – MULIG Å FINNE PASSORD	14
Sårbarhet 3 – XSS	15
Sårbarhet 4 – OPENSSSH VERSJON 7.9 UTDATERT	18
Sårbarhet 5 – SVAKE ALGORITMER VED OPENSSSH VER 7.9	19
Sårbarhet 6 - SCRF	21
Sårbarhet 7 – TLS KONFIG – TLS 1.0 OG 1.1 UTDATERT	21
Sårbarhet 8 – TLS KONFIG – STØTTER SVAKE CIPHERS	22
Sårbarhet 9 - CLICKJACKING	23
Sårbarhet 10 – SENSITIV FIL TILGJENGELIG PÅ SIDEN	25
Sårbarhet 11 – DNS BIND VERSJON 9.11 UTDATERT	26
Sårbarhet 12 – SQL ERROR VISES PÅ SIDEN	27
Sårbarhet 13 – X-CONTENT-TYPE-OPTIONS HEADER MANGLER	28
Sårbarhet 14 – COOKIES WITHOUT HTTPONLY FLAG	29
Sårbarhet 15 – PORT 79 FINGER ÅPEN	29
Sårbarhet 16 – PORT 37 TIME ÅPEN	30
Sårbarhet 17 – HSTS HEADER MANGLER	31



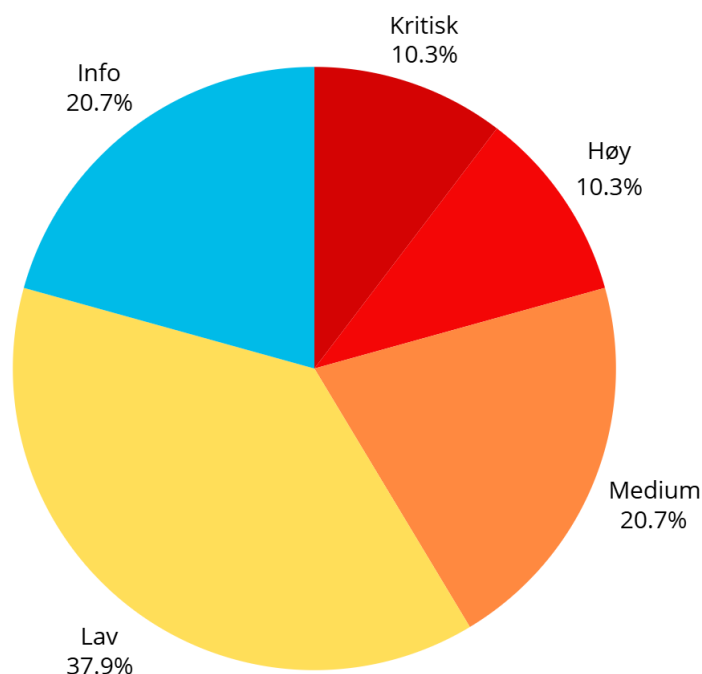
Sårbarhet 18 – COOKIES WITHOUT SECURE FLAG.....	31
Sårbarhet 19 – TLS KONFIG - RENEGOTIATION TILLATER DOS ANGREP.....	32
Sårbarhet 20 – SERVER LEAKS VERSION INFORMATION VIA SERVER http RESPONSE HEADER.....	32
Sårbarhet 21 – PORT 139 ÅPEN	33
Sårbarhet 22 – AUTOFYLL PASSORD TILLATT.....	34
Sårbarhet 23 – BASIC AUTHENTICATION	34
Informasjon 1 – Falsk positiv SSTI.....	35
Informasjon 2 - FALSK POSITIV IP-ADRESSE I SERVER HEADER.....	36
Informasjon 3 – PORT 445 MICROSOFT-DS ÅPEN.....	36
Informasjon 4 – PORT 9999 ABYSS ÅPEN.....	37
Informasjon 5 – PORT 9 ÅPEN.....	37
Informasjon 6 – PORT 13 ÅPEN.....	37

Executive Summary

Dette er en sikkerhetsrevisjon av selskapet Boris Lockpicks' nettside. Det ble utført en white box test av HKSecurity, kandidat 50200 Cybersikkerhetsstudent ved Kristiania i en periode på 3 uker fra 1.11.2024 til 22.11.2024. Med ulike tester, fant vi ut at denne nettsiden har flere sårbarheter inkludert kritisk risiko for SQL injection, XSS og mulighet for å finne passord. Serveren har også mange utdaterte systemer og mangler flere sikkerhetskomponenter i koden.

Fremover bør dere fokusere på å oppdatere alle systemene og servere deres til de nyeste versjonene. Utdaterte systemer er en gjenganger hos BL og er essensielt for å opprettholde sikkerheten helhetlig. Dere burde vurdere å utlyse en stilling som systemadministrator som jevnlig oppdaterer systemene og passer på at unødvendige porter lukkes. I tillegg anbefaler jeg å sette opp et SIEM, slik at selskapet er forberedt dersom det skulle oppstå sikkerhetshendelser.

Anbefaler dere også på det sterkeste å gå grundig gjennom koden for å legge til nødvendige sikkerhets headers og protokoller, samt sanitere input. For senere utvikling av nettside, bør dere sette klare rammeverk for utvikling av sikkerhetskomponenter, slik at kritiske sårbarheter som SQLI blir unngått.



Konfidensialitetsavtale

Dette er en sikkerhetsrevisjon gjennomført av Kandidat 50200 I emnet etisk hacking/ETH2100. Dette dokumentet er konfidensielt og klientens eiendom og må ikke benyttes utover det som er avtalt med klienten i hovedkontraktsdokumentet BL-453-654 Security Commercial Agreement.

Ansvarsfraskrivelse

Formålet med dette dokumentet er å informere BL om sårbarheter ved deres web applikasjon og som en sikkerhetsveiledning basert på tidspunktet web applikasjonen ble testet på. Nettsiden kan siden ha blitt forandret, patchet eller fått nye sårbarheter og anbefalingene i denne rapporten kan utdateres ved senere tid. Derfor er det ikke anbefalt å bruke denne rapporten som et utgangspunkt i senere tid. Vi kan ikke garantere at rapporten tar for seg alle sårbarheter ved serveren til BL. Rapporten er ment for utvikling og ikke for utnytting. Rapporten skal kun brukes til denne hensikten.

Omfang

Omfanget baserer seg på begrensninger satt av BL i enighet med HKSecurity. BL gir HKSecurity tillatelse til å utføre tester på vegne av selskapet. Dette gjelder all testing utenom social engineering, open source intelligence og testing av selskapet Eastwill security. Det at siden har en self-signed certificate skal ikke rapporteres som en sårbarhet. BL vil at alt på serveren skal testes.

Versjonslogg

Dokument navn: 2024-10-12 BL PEN-T ver 1.0

Versjon	Dato	Endring	Utført av
1.0	21.11.2024	Første rapport	Kandidatnr: 50200

Forkortelser

Forkortelser benyttet i dette dokumentet:

PEN-T – penetrasjonstest

BL – Boris' Lockpicks

SQLI – SQL injection

XSS – Cross Site Scripting

SSTI – Server side template injection

SSH – Secure shell

DNS – domain name system

Gjennomførelse

Testen retter seg mot nettsiden til BL.

Deltagere

Testen ble utført av Kandidat 50200, Cybersikkerhets-student på Kristiania.

Verktøy

Verktøy benyttet i denne analysen:

Verktøy	Versjon	Kommentar
Lenovo Yoga PC		
VMware	17.6.1	
Nikto	2.5.0	I kali linux terminal
OWASP ZAP	2.15.0	
Nessus	10.8.3	Trial version
Kali Linux	2024.2	I VMware
NMAP	7.94SVN	I kali linux terminal
Sslyze	6.0.0	I kali linux terminal
SQLmap	1.8.9#stable	I kali linux terminal
tplmap	0.5	I kali linux terminal
hydra	9.5	I kali linux terminal

Tidslinje

Versjon 1.0 – 01.11.2024 – 22.11.2024

Testtype

White box test med tilgang til kildekode og server.

Oversikt over sårbarheter

Sårbarhetsvurdering

Level	Score	Beskrivelse
Kritisk	9 - 10	Kritiske sårbarheter stiller umiddelbar risiko for serveren og konsekvensene av utnyttelse kan føre til alvorlige skader på selskapet. Kritiske sårbarheter må fikses opp I umiddelbart.
Høy	7 - 9	Høye sårbarheter stiller umiddelbar risiko og konsekvensene av utnyttelse kan føre til store tap for selskapet. Høye sårbarheter skal prioriteres.
Medium	4 - 7	Middels sårbarheter stiller risiko for selskapet men er vaskeligere for angriper å utnytte fullstendig. Disse sårbarhetene skal fikses når det er mulighet for det
Lav	1 - 4	Lave sårbarheter stiller liten risiko for selskapet og er vaskelige å utnytte. Sårbarhetene skal fikses hvis det er mulighet for det.
Informasjon	0	Informasjon stiller ingen risiko for selskapet og er bare ment for å informere.

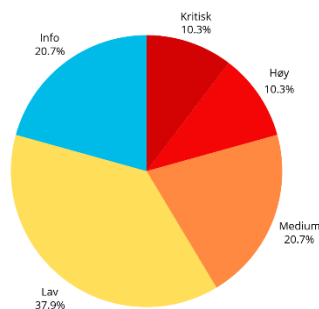
Risikomatrise

Alle sårbarhetene er vurdert etter sannsynligheten for utnyttelse for selskapet målt mot alvorligheten av konsekvensene ved utnyttelse.

Sannsynlighet	Svært høy	Middels	Høy	Høy	Svært høy
	Høy	Middels	Middels	Høy	Høy
	Moderat	Lav	Middels	Middels	Høy
	Lav	Lav	Lav	Middels	Middels
		Lav	Moderat	Høy	Svært høy
		Konsekvens			

Sårbarhetstabell

Kategori	Sårbarhet	Risiko
Sårbarhet 1	SQLI	Kritisk
Sårbarhet 2	MULIG Å FINNE PASSORD	Kritisk
Sårbarhet 3	XSS	Kritisk
Sårbarhet 4	OPENSSSH 7.9 UTDATERT	Høy
Sårbarhet 5	SSH SVAKE ALGORITMER	Høy
Sårbarhet 6	CSRF 7.5	Høy
Sårbarhet 7	TLS KONFIG – UTDATERTE TLS VERSJONER	Medium
Sårbarhet 8	TLS KONFIG – STØTTER SVAKE CIPHERS	Medium
Sårbarhet 9	CLICKJACKING	Medium
Sårbarhet 10	TILGJENGELIG FIL SERVERINFORMASJON	Medium
Sårbarhet 11	DNS BIND VERSJON 9.11 UTDATERT	Medium
Sårbarhet 12	REDIR TIL SQL SYNTAKS ERROR	Medium
Sårbarhet 13	X-CONTENT-TYPE-OPTIONS HEADER MANGLER	Lav
Sårbarhet 14	COOKIES WITHOUT HTTPONLY FLAG	Lav
Sårbarhet 15	PORT 79 FINGER ÅPEN	Lav
Sårbarhet 16	PORT 37 TIME ÅPEN	Lav
Sårbarhet 17	HSTS HEADER MANGLER	Lav
Sårbarhet 18	COOKIES WITHOUT SECURE FLAG	Lav
Sårbarhet 19	TLS KONFIG – STØTTER DOS ANGREP	Lav
Sårbarhet 20	SERVER INFO I HTTP RESPONSE HEADER	Lav
Sårbarhet 21	PORT 139 ÅPEN	Lav
Sårbarhet 22	AUTOFYLL PASSORD TILLATT	Lav
Sårbarhet 23	BASIC AUTHETICATION	Lav
Informasjon 1	Falsk positiv – SSTI	Info
Informasjon 2	Falsk positiv – ip adresse i server header	Info
Informasjon 3	Port 445 microsoft-ds åpen	Info
Informasjon 4	Port 9999 abyss åpen	Info
Informasjon 5	Port 9 åpen	Info
Informasjon 6	Port 13 åpen	Info



Detaljert beskrivelse av funn

Sårbarhet 1 – SQLI

RISIKO: kritisk

Beskrivelse

SQLI angrep er angrep hvor man kan få tilgang til databasen ved å skrive sql spørringer til siden uten at disse kontrolleres av serveren, og kjøres.

Oppdagelse

I Owasp zap fikk jeg opp at SQLI kunne være mulig, grunnet feilmeldingen som ble returnert fra siden «You have an error in your SQL syntax» sannsynligvis fordi zap prøvde å skrive inn en SQL query i et input-felt, hvor syntaksen ble feil. Denne feilmeldingen tilsier at SQLI kan være mulig, ettersom at den reagerer på syntaksen.

Fikk også opp i Nessus scans at den hadde fått opp en feilmelding fra databasen når den hadde prøvd å sende en sql query. Samme som i Owasp zap.

Valgte å teste dette manuelt i mypage_update under adresse feltet og ble først omdirigert til en siden med samme feilmelding om at det var feil i SQL syntaksen, samme som Owasp zap hadde fått. Her forteller den meg også hvor i syntaksen jeg har feil. Fikk også opp feilmelding som ga informasjon om databasen og hvordan queries sendes, inkludert navn på databasen, om tabellen eksisterer.

Etter dette kjørte jeg SQLmap på ipadressen til siden og fikk opp at error-based SQLI, boolean-based blind SQLI, Time-based blind SQLI og Union query injection var mulig i ID parameteren på sidene store_viewdetails.php, store_addtobasket.php og searchresults.php. Kjørte også SQLmap som hentet ut informasjon om databasen og tabeller. Her fikk jeg oversikt over alle brukere og passord hashene i tillegg til at den løste noen av hashene.

Konsekvenser

Det at databasen sender feilmeldinger og viser til hvor i syntaksen det er feil og hvordan SQL spørringer sendes, gjør det mye enklere for en angriper å finne ut av hvordan han skal skrive SQL queries. Den forteller også om databasen i enkelte spørringer, sånn som at tabeller ikke eksisterer og navn på database, noe som gjør det veldig enkelt å kartlegge hvordan databasen er satt opp. Dette kan en hacker bruke til å hente ut all dataen fra databasen, inkludert brukernavn og passord på brukere, endre data i databasen, legge inn nye dataer/tabeller, slette data. Sensitiv informasjon kan brukes for å logge seg inn på brukere, endre bestillinger, hente ut kortinformasjon på de forskjellige brukerne og utnytte de.

Løsning

I stedet for å direkte legge bruker input i SQL-spørringer i koden, anbefales det å bruke prepared statements. Prepared statements vil først forberede en SQL-spørring med «?» i stedet for brukerinputet direkte. Brukerinputet lagres som parametere. Parameteren bindes til sql spørringen og kjøres. Dette gjør at bruker inputet alltid tolkes som data og ikke som kode. Dette burde gjøres på alle SQL-spørringer med input, i koden.

Selv om dette er en god prosedyre for å forebygge SQLI finnes det andre tiltak som stored procedures og input validering. Du kan lese mer om forebygging av SQLI via denne linken:

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Anbefaler også å sette opp en WAF som automatisk fanger opp og blokkerer SQLI forsøk.

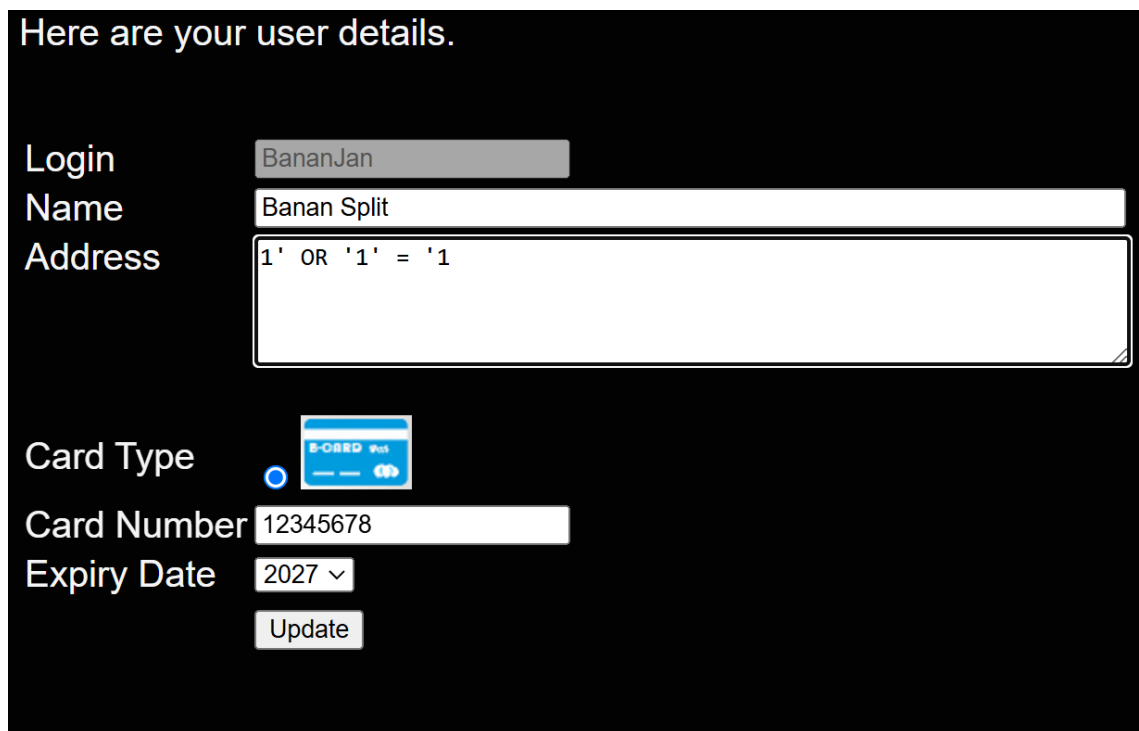
Dokumentasjon

En intern feil i SQL, ukjent type

```
SQL = UPDATE customer SET name='Banan Split',address='1' OR (SELECT 1 FROM Users WHERE username='admin')='1',cardnumber='12345678',expiryyear=2027 where uid=11
```

```
ERROR mysqli_query() failed [1146] Table 'borislockpicks.Users' doesn't exist
```

Figur 1.1: Viser at databsen kan nås og navnet på databasen



Here are your user details.

Login	BananJan
Name	Banan Split
Address	1' OR '1' = '1
Card Type	
Card Number	12345678
Expiry Date	2027 ▾
	<button>Update</button>

Figur 1.2: SQL syntaksen jeg skrev inn i adressefelt

Here are your user details.

Login

Name

Address

Card Type ☒ 

Card Number

Expiry Date

Data saved...

Figur 1.3: Adressefeltet endret seg til 1 etter at jeg trykket på update, noe som tilsier at koden ble kjørt på serveren.

```
[07:36:45] [INFO] the back-end DBMS is MySQL
web server operating system: Linux
back-end DBMS: MySQL ≥ 5.0 (MariaDB fork)
[07:36:45] [INFO] fetching database names
[07:36:45] [INFO] retrieved: 'information_schema'
[07:36:45] [INFO] retrieved: 'borislockpicks'
[07:36:46] [INFO] retrieved: 'mysql'
[07:36:46] [INFO] retrieved: 'performance_schema'
available databases [4]:
[*] borislockpicks
[*] information_schema
[*] mysql
[*] performance_schema

[07:36:46] [INFO] fetching tables for database: 'borislockpicks'
[07:36:46] [INFO] retrieved: 'logon_sessions'
[07:36:47] [INFO] retrieved: 'admin_sessions'
[07:36:47] [INFO] retrieved: 'lpbasket_entry_global'
[07:36:47] [INFO] retrieved: 'employee'
[07:36:47] [INFO] retrieved: 'products'
[07:36:47] [INFO] retrieved: 'customer'
[07:36:47] [INFO] retrieved: 'borislpbasket'
Database: borislockpicks
[7 tables]
+-----+
| admin_sessions |
| borislpbasket  |
| customer       |
| employee       |
| logon_sessions |
| lpbasket_entry_global |
| products       |
+-----+
```

Figur 1.4: SQLmap som henter ut informasjon om databasen.



Database: borislockpicks
Table: customer
[7 entries]

uid	login	name	logins	pwhash	address	cardnumber	expiryyear
1	bengt	Bengt Ostby	429136	84d961568a65073a3bcf0eb216b2a576 (superman)	Hoyskolen Kristiania\r\n0999 Oslo	12312312	2023
2	anne	Anne Holm	0	a0dff60cf804e30e76745e734571d1c3	Hoyskolen Kristiania\r\n0999 Oslo	11563300	2026
5	karina	Karina Bjork	0	308e1920c81ba72b0788839184bea5ed	Oslogate 42\r\n0101 Oslo	98373988	2027
8	stian	Stian Kvals	0	9e43731b669b2e0f6accfc1881615efa	Gateadressen 12\r\n3299 Huttiheita	45645645	2024
9	navn	Navn Navnessen	0	dd95e6ea0c2ffb0cc0d6f6549dfd756 (mittpassord)	Standardveien 99\r\n9999 Usett	01917488	2027
10	NotReal1	NotRealName	0	9e91090096ceee0aae5ddae65d7e3f5c	NotRealAddress 5	12345678	2026
11	BananJan	ZAP	0	dad998c462994007c076f52a21544f74	688 Zaproxy Ridge	12345678	2027

Figur 1.5: Tabellen customers henta ut fra databasen borislockpicks, med brukernavn, passordhash og enkelte passord.

Here are your user details.

Login:

Name:

Address:

Card Type: ☒ 

Card Number:

Expiry Date:

Figur 1.6: Logget inn på bruker bengt.

Login:

Name:

Address:

Card Type: ☒ 

Card Number:

Expiry Date:

Figur 1.7: Logget inn på navn.

```
$result = _sqlqueryfunc_eh("SELECT * FROM products WHERE uid=".$_GET["id"]);  
$item = mysqli_fetch_array($result);
```

Figur 1.8: fra `store_viewdetails.php` i kildekoden hvor man ser at «id» legges direkte inn i SQL-spørringen.

Sårbarhet 2 – MULIG Å FINNE PASSORD

RISKO: kritisk

Beskrivelse

Det er mulig å finne passord og knekke passord til brukerne på nettsiden. Siden webserveren er sårbar for SQLi er det mulig å hente ut databasetabellen customers, som er en tabell med informasjon om alle brukere, samt hashene til passordene deres. Passord er kryptert med hashing md5 uten salt. md5 er en utdattert og usikker hashing algoritme. Den er mye enklere å dekryptere enn andre mer moderne hashing algorithmer og uten en salt kan de knekkes på sekunder. Dette er fordi md5 baserer seg på 128 bit i stedet for 256 bit som mange andre hashing algoritmer bruker.

Oppdagelse

Jeg utnyttet SQLi sårbarheten for å hente ut tabellen med informasjon om brukerne. Da fikk jeg opp alle brukernavn og hashene til passordene. Siden md5 hashing blir brukt uten salt var det veldig enkelt å dekryptere disse hashene. Dette gjorde jeg gjennom sqlmap.

Konsekvenser

Angriper kan få tilgang til brukere

Løsning

Løsningen vil være å kryptere passordene med en sterkere hash som bcrypt og salte hashet, slik at det blir mye vanskeligere å dekryptere. I tillegg må serveren sikres mot SQLi. Jeg anbefaler også å bruke sterkere passord ettersom at passordene er veldig svake og kan finnes i passordfiler som brukes til å knekke passord.

Dokumentasjon

```
d pwhash='".md5($_POST["password"])."');
```

Figur 6.1: bilde fra kildekoden hvor hashing md5 brukes.

```
Database: borislockpicks
Table: customer
[7 entries]
```

uid	login	name	logins	pwhash	address	cardnumber	expiryyear
1	bengt	Bengt Ostby	429136	84d961568a65073a3bcf0eb216b2a576 (superman)	Hoyskolen Kristiania\r\n0999 Oslo	12312312	2023
2	anne	Anne Holm	0	a0dff60cf804e30e76745e734571d1c3	Hoyskolen Kristiania\r\n0999 Oslo	11563300	2026
5	karina	Karina Bjork	0	308e1920c81ba72b0788839184bea5ed	Oslogate 42\r\n0101 Oslo	98373988	2027
8	stian	Stian Kvals	0	9e43731b669b2e0f6accfc1881615efa	Gateadressen 12\r\n3299 Huttuheita	45645645	2024
9	navn	Navn Navnessen	0	dd95e6ea0c2ffb0cc00d6f6549dfd756 (mittpassord)	Standardveien 99\r\n9999 Usett	01917488	2027
10	NotReal1	NotRealName	0	9e91090096ceee0aae5ddae65d7e3f5c	NotRealAddress 5	12345678	2026
11	BananJan	ZAP	0	dad998c462994007c076f52a21544f74	688 Zaproxy Ridge	12345678	2027

Figur 6.2: bilde fra en sqlmap scan hvor jeg fikk tilgang til tabellen med brukere og passordhasher.

Sårbarhet 3 – XSS

RISIKO: HØY

Beskrivelse

Gjennom flere scans og manuell testing oppdaget jeg at siden er sårbar for Cross Site Scripting også kjent som XSS angrep. XSS angrep er et klient-side angrep hvor angriperen injiserer skadelig kode inn på siden, som deretter vil kjøres på brukerens side. Dette kan brukes til å for eksempel å hente ut sensitiv informasjon om brukeren. Siden er sårbar for både Reflected XSS, Stored XSS. Reflected XSS oppstår når koden som injiseres sendes til serveren og serveren reflekterer koden tilbake til siden uten å godkjenne det. Stored XSS skjer når kode blir lagret på serveren og senere kjøres som en del av dataen.

Oppdagelse

I ulike scans som Owasp, nikto og nessus fikk jeg opp flere varsler om at siden var sårbar for XSS på flere deler av siden. I tillegg fikk jeg opp at Content Security policy headeren ikke er tatt i bruk, en parameter som forhindrer injeksjonsangrep slik som XSS.

Jeg fikk også opp noen andre sårbarheter som ikke direkte tillater XSS men som kan videre føre til XSS angrep. For eksempel at cookies ikke har SameSite Attribute eller httponly flagg og at X-content-Type-Options header ikke er til stede.

Alle disse funnene indikerer at siden er sårbar for XSS angrep, derfor ville jeg teste om den faktisk var det manuelt.

Dette gjorde jeg først ved å sjekke de ulike input feltene, men her fikk jeg ikke noe resultat. Deretter sjekket jeg om det var mulig å injisere javascript kode inn i URL-en, ved å skrive `<script>alert(«Siden er sårbar for XSS!!!»)</script>` Da kjørte koden jeg skrev inn og jeg fikk opp en alert på siden. Dette beviser at siden er sårbar for reflected XSS, som vil si at koden sendes til serveren og serveren reflekterer koden ut til siden igjen uten å godkjenne det.

Jeg skrev også inn `<script>alert(«XSS!»)</script>` i adressefeltet for brukeren jeg lagde og la merke til at noen ganger når siden for betaling ble sendt, fikk jeg opp alerten med XSS! Fordi siden ville outputte adressen til brukeren. Dette beviser at siden er sårbar for stored XSS, som er den mest alvorlige typen XSS angrep.

Konsekvenser

Cross Site Scripting er et klient-side angrep som rammer kunden eller brukeren av siden, hvor angriper kan skrive inn javascript kode i et input felt for å manipulere siden. Dette kan føre til at angriper kan få tilgang til sensitiv informasjon, ta over brukeren, injisere virus osv. Det finnes mange konsekvenser for disse typer angrep, siden angriper kan skrive nesten hva slags kode han vil. Hovedsakelig vil dette gå ut over bruker av siden og kan ha alvorlige konsekvenser for kundene. Dette kan også gi andre indirekte konsekvenser som f.eks skade av omdømme til selskapet og føre til juridiske sanksjoner og økonomiske tap.

Løsning

Det er finnes mange tiltak som kan gjøre siden sikker mot XSS angrep og en kombinasjon av flere av de vil være optimalt.

Først og fremst anbefaler jeg å validere brukerinput og output som sendes til og fra serveren. Dette gjøres ved å blant annet sette `htmlspecialchars` og `strip_tags`. Disse fjerner eller endrer spesielle tegn som kan tolkes som kode, slik at serveren ikke leser de som kode.

I tillegg anbefaler jeg å sette en Content Security Policy header. Dette er en http-parameter som forhindrer injiserings angrep slik som XSS og clickjacking. For at den skal hindre XSS skrives den inn i koden slik:

Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none';

Dette gjør at siden kun tillater ulike ressurser fra samme kilde og blokkerer objekter fra å bli lastet inn, samt forhindrer clickjacking - at siden kan bli lastet inn i en ramme eller et objekt.

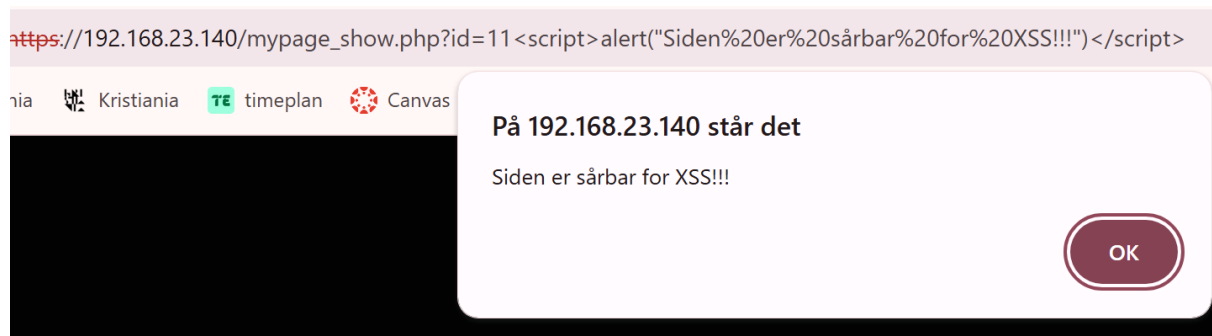
For å bedre sikre mot XSS angrep ville jeg også satt SameSite attribute og httpOnly flagg på cookies og sette en X-Content-Type-Options header.

Mange av systemene som brukes er utdaterte så jeg anbefaler å oppdatere, ettersom at mange av sårbarhetene blir løst i oppdateringer. Men aller helst vil jeg anbefale å ta i bruk nyere systemer,

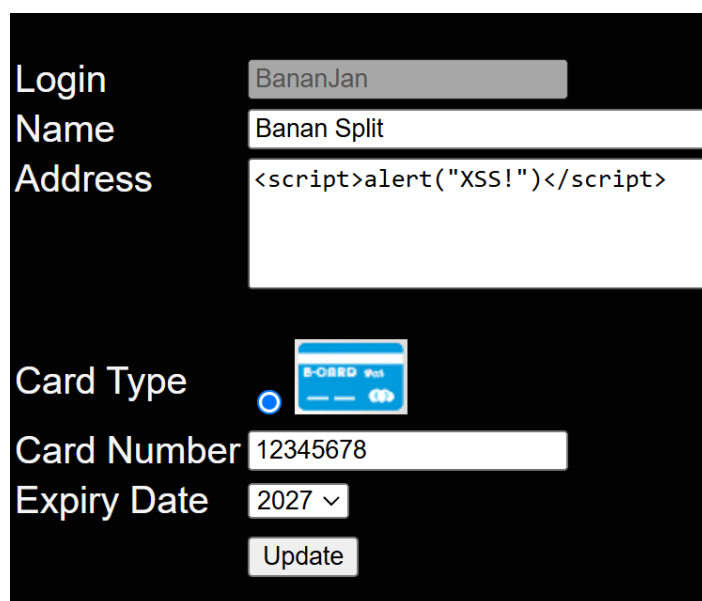
Dette som fks CMS systemet mambo er veldig utdatert og kjent for å være sårbar for XSS, derfor ville system som joomla eller wordpress egne seg mer.

Ta i bruk en web application firewall (WAF). Dette er en brannmur for webapplikasjoner som beskytter siden ved å filtrere ut ressurser som kan være skadelige for siden. Den virker som en beskyttelse mellom siden og internettet. Den hjelper til med å beskytte mot angrep som XSS, SQLI og lignende.

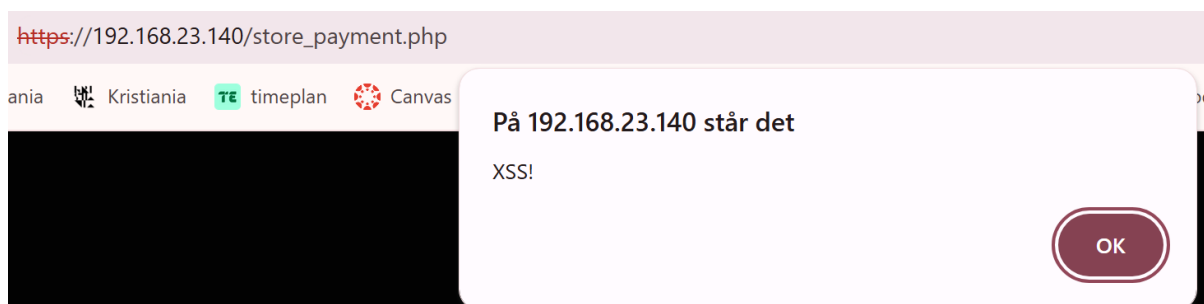
Dokumentasjon



Figur 2.1: Reflected XSS. Her skrev jeg inn js kode inn i URL på my page, som kjørte.



Figur 2.2: Stored XSS. Brukerensiden til brukeren min der jeg lagret kode inn i adressefeltet.



Figur 2.3: Stored XSS. Alerten jeg fikk opp når siden skulle sende inn adressen til brukeren på payment siden.

Sårbarhet 4 – OPENSSH VERSJON 7.9 UTDATERT

RISIKO: Høy

Beskrivelse

OpenSSH versjon 7.9 er en utdatert versjon med flere kjente sårbarheter som kan utnyttes av angripere.

Oppdagelse

SSH-audit scan

Oversikt over sårbarheter

CVE	TYPE	CVSS	VERSJONER
CVE-2021-41617	Privilege escalation	7.0	6.2 – 8.8

Beskrivelse

Sshd konfigurerer ikke grupper på riktig måte. Dette kan brukere utnytte til å få tilgang til høyere brukerprivilegier gjennom å bruke noen ikke standardiserte konfigurasjonsinnstillinger.

CVE	TYPE	CVSS	VERSJONER
CVE-2020-15778	Command injection	7.8	≤ 8.3

Beskrivelse

Sårbarhet i secure copy kommandoen som brukes til å sende sikre filkopier mellom datamaskin og servere i SSH. Dette kan føre til at personer med tilgang til SSH, kan sende kommandoer til systemet gjennom scp-kommandoen når man overfører filer ved å legge til spesifikke tegn. Dette kan føre til at en angriper sender sin egen kode til serveren.

CVE	TYPE	CVSS	VERSJONER
CVE-2016-20012	Enumerate usernames	5.3	≤ 8.7

Beskrivelse

Angripere kan sjekke om bestemte brukernavn i kombinasjon med en public key er kjent for SSH serveren. Hvis kombinasjonen finnes på serveren, vil den gi en respons. Dette vil fortelle angriperen noe om brukerne.

Løsning

Oppdater OpenSSH til en nyere versjon, OpenSSH 9.9 er den nyeste versjon tilgjengelig på <https://www.openssh.com/>

Sårbarhet 5 –SVAKE ALGORITMER VED OPENSSH VER 7.9

RISIKO: Høy

Beskrivelse

Mange av algoritmene som brukes i OpenSSH 7.9 er sårbare og burde enten endres eller fjernes.

Oppdagelse

Disse sårbare algoritmene ble oppdaget gjennom en Ssh-audit verbose scan.

Oversikt over sårbare algoritmer

Key exchange algoritmer (KEX)

Algoritme	sårbarhet	Anbefaling
Ecdh-sha2-nistp256	Mistanke om NSA bakdør	Fjern denne algoritmen
Ecdh-sha2-nistp384	Mistanke om NSA bakdør	Fjern denne algoritmen
Ecdh-sha2-nistp521	Mistanke om NSA bakdør	Fjern denne algoritmen
Diffie-hellman-group14-sha256	2048-bit modul har begrenset styrke på 112-bit	Fjern denne algoritmen
Diffie-hellman-group14-sha1	Bruker ødelagt hash algoritme (SHA1)	Fjern denne algoritmen

Host-key algoritmer (KEY)

Algoritme	Sårbarhet	Anbefaling
Rsa-sha2-512 (2048-bit)	2048-bit modul har begrenset styrke på 112-bit	Endre denne modul størrelse til 3072-bits eller større
Rsa-sha2-256 (2048-bit)	2048-bit modul har begrenset styrke på 112-bit	Endre modul størrelse til 3072-bits eller større
Ssh-rsa (2048-bit)	Bruker ødelagt hash algoritme (SHA1)	Fjern denne algoritmen
Ssh-rsa (2048-bit)	2048-bit modul har begrenset styrke på 112-bit	Fjern denne algoritmen
Ecdsa-sha2-nistp256	Mistanke om NSA bakdør	Fjern denne algoritmen
Ecdsa-sha2-nistp256	Privat nøkkel kan bli oppdaget	Fjern denne algoritmen

Ciphers/krypterings algoritmer

algoritme	sårbarhet	Anbefaling
Chacha20-poly1305@openssh.com	Sårbar for terrapin attack (CVE-2023-48795)	Fjern denne algoritmen

MAC algoritmer

algoritme	sårbarhet	Anbefaling
Umac-64-et@openssh.com	Bruker 64-bit tag som gir begrenset sikkerhet	Fjern denne algoritmen
Hmac-sha1-et@openssh.com	Bruker ødelagt hash algoritme	Fjern denne algoritmen
Umac-64@openssh.com	Bruker encrypt-and-mac som er mindre sikker enn encrypt-then-mac	Fjern denne algoritmen
Umac-64@openssh.com	Bruker 64-bit tag som gir begrenset sikkerhet	Fjern denne algoritmen
Umac-128@openssh.com	Bruker encrypt-and-mac som er mindre sikker enn encrypt-then-mac	Fjern denne algoritmen
Hmac-sha2-256	Bruker encrypt-and-mac som er mindre sikker enn encrypt-then-mac	Fjern denne algoritmen
Hmac-sha2-512	Bruker encrypt-and-mac som er mindre sikker enn encrypt-then-mac	Fjern denne algoritmen
Hmac-sha1	Bruker ødelagt hash algoritme (SHA1)	Fjern denne algoritmen
Hmac-sha1	Bruker encrypt-and-mac som er mindre sikker enn encrypt-then-mac	Fjern denne algoritmen

Konsekvenser

På grunn av disse svake algoritmene, kan en angriper potensielt utføre brute-force-angrep, spoofing osv.

Løsning

Her anbefales det å fjerne eller endre algoritmene over. Det å oppdatere til en nyere versjon av openSSH vil gjøre dette automatisk. Den nyeste versjonen tilgjengelig, OpenSSH 9.9, er tilgjengelig på <https://www.openssh.com/>

Sårbarhet 6 - CSRF

RISIKO: Høy

Beskrivelse

CSRF er et angrep der angriper får en aktiv bruker til å utføre en handling uten at personen vet det selv. Angriper lurer brukeren til å sende en http-forespørsel som utfører en handling på vegne av brukeren. Dette kan for eksempel være en link som angriper har sendt til brukeren som utfører en handling på vegne av brukeren, når personen trykker på linkene.

Oppdagelse

Denne sårbarheten dukket opp i en owasp zap scan. Fikk også opp at cookies mangler samesite attribute. For å nærmere undersøke dette, gikk jeg gjennom kildekoden hvor jeg så at sidene manglet CSRF-tokens, som validerer forespørsler som legitime. Siden CSRF-tokens også mangler på siden, vil den være sårbar for CSRF angrep.

Konsekvenser

Angriper kan utføre handlinger på vegne av en autorisert bruker. Dette kan for eksempel være bankoverføringer, endring av kontoinformasjon, varekjøp osv.

Løsning

Sette anti-csrf tokens på sidene og samesite attribute på cookies.

Sårbarhet 7 – TLS KONFIG – TLS 1.0 OG 1.1 UTDATERT

RISIKO: Høy

Beskrivelse

TLS 1.0 og 1.1 støttes, selv om disse er utdaterte TLS-versjoner.

Oppdagelse

Sslyze av ip-adressen

Konsekvenser

TLS 1.0 og 1.1 er utdaterte og har kjente sårbarheter. De er sårbare for BEAST og POODLE som er angrep hvor angriper kan dekryptere og få tilgang til sensitiv data.

Løsning

Deaktiver TLS 1.0 og 1.1 og konfigurer serveren til å ta i bruk 1.2 og 1.3 som laveste versjon.

Sårbarhet 8 – TLS KONFIG – STØTTER SVAKE CIPHERS

RISIKO: Høy

Beskrivelse

TLS 1.2 støtter svake cipher suites som ikke har forward secrecy. Forward Secrecy passer på at nøklene som brukes i en session ikke brukes på nytt og at de ikke kan knekkes. I tillegg har RSA key nøkkelstørrelse 1024 som er for liten og burde ha minst nøkkelstørrelse 2048.

Ciphers uten forward secrecy:

- TLS_RSA_WITH_AES_256_CBC_SHA
- TLS_RSA_WITH_AES_128_CBC_SHA
- TLS_DHE_RSA_WITH_AES_256_CBC_SHA256

Oppdagelse

Sslyze av ip adressen

Konsekvenser

Siden disse ikke bruker Forward Secrecy kan nøkler bli brukt til å dekryptere data fra tidligere sessions.

Løsning

Fjern cipher suites som ikke bruker forward secrecy og øk nøkkelstørrelsen til diffie-hellman til minst 2048. Pass på at TLS bruker sterke cipher suites under konfigurering av TLS.

Sårbarhet 9 - CLICKJACKING

RISIKO: Medium

Beskrivelse

X-Frame-Options header er ikke satt og siden er sårbar for clickjacking. En X-Frame-Options header begrenser hvorvidt en nettleser kan laste opp siden i frames eller objekter i scripts. Hvis en side kan bli lastet opp i et script kan en angriper skrive kode som får knapper på siden til å gjøre noe annet enn det de ser ut til å gjøre. Dette kalles clickjacking.

Oppdagelse

I en nikto scan av nettsidens Ip-adresse og en owasp zap active scan fikk jeg opp at anti-clickjacking headeren X-Frame-Options ikke var satt. Siden dette er en header som beskytter nettsider mot clickjacking, ville jeg teste ut om det faktisk gikk an å clickjacke siden.

I tillegg fikk jeg opp at CSP headeren heller ikke tas i bruk, i en owasp zap scan. Denne headeren beskytter mot mange ulike injiseringsangrep, blant annet clickjacking.

Jeg testet siden for clickjacking ved å kjøre en enkel test hvor jeg skrev en html-kode for en ny nettside og lastet BL nettsiden inn i en frame på siden min. Deretter åpnet jeg siden i en nettleser, hvor jeg så at BL var lastet opp og fungerte akkurat som på deres egen side.

Konsekvenser

Clickjacking er et klient-side angrep. Her kan angriper legge på et element over et annet element som bruker ikke vil legge merke til engang, men som kan bety at man bekrefter transaksjoner og deling av informasjon man ikke vet om.

Siden BL har en nettbutikk på siden og brukere lagrer opplysninger om kort-informasjon, vil den mest kritiske konsekvensen være at en angriper kan utnytte denne sårbarheten for å ta i bruk kort hos en bruker eller at en bruker bekrefter å sende kortinformasjon til angriper helt uvitende ved å for eksempel trykke på update-knapp etter å ha oppdatert kontoinformasjon.

Angriper kan legge til elementer på siden som bruker av siden uvitende trykker på eller fyller ut. Disse elementene kan brukes til å hente ut sensitiv informasjon om pålogging, kontoer, kort. Denne informasjonen kan brukes til for eksempel utpressing, komprimering av kontoer, datalekkasje, identitetstyveri og mye mer.

I tillegg til at dette er veldig alvorlige konsekvenser, vil nettsiden også muligens bli oppfattet som utrygg, miste sin pålitelighet og få negativ omtale hos kunder.

Løsning

Løsningen vil være å legge til headers som beskytter siden mot clickjacking – både X-Frame-Options header og CSP header.

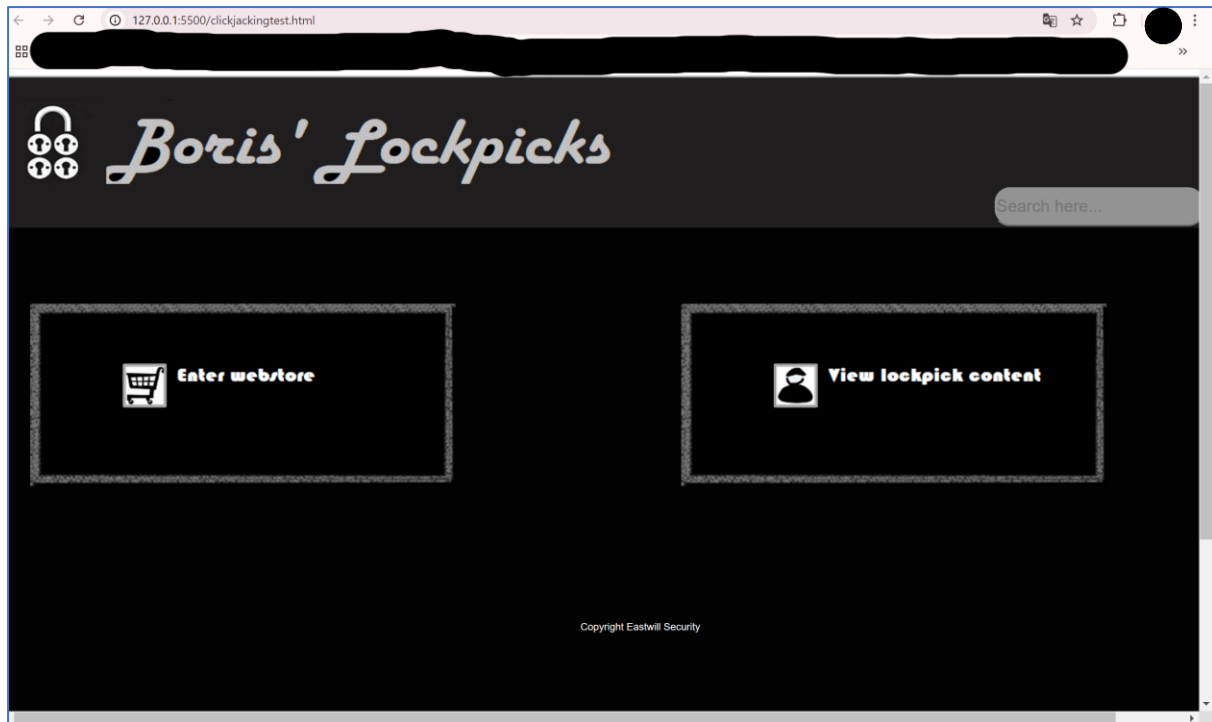
X-Frame-Options headeren beskytter siden mot clickjacking attacks ved å indikere om siden kan lastes inn i en frames eller objekter i andre siders script. Man kan velge hvorvidt man vil at siden skal kunne lastes opp i frames. Den kan settes til 3 verdier: DENY, SAMEORIGIN og ALLOW-FROM uri. DENY forhindrer at nettsiden kan lastes opp i frames i det hele tatt. SAMEORIGIN tillater opplasting i frames, men bare hos nettsider som har samme domene. ALLOW-FROM uri tillater opplasting i frames på sider med godkjente domener, denne brukes sjeldent lenger, ettersom at CSP headeren er et bedre alternativ. X-Frame-Options headeren legges direkte inn i koden.

CSP headeren beskytter også mot clickjacking og jeg anbefaler også å ta i bruk denne. X-Frame-Options holder på å bli utdatert og erstattet med frame-ancestors i CSP headeren.

Dokumentasjon

```
<> clickjackingtest.html >  html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>clickjacking test</title>
7  </head>
8  <body>
9      <iframe src="http://192.168.23.141" width="1500" height="1000"></iframe>
10 </body>
11 </html>
```

Figur 5.1: kode som lager en html side med BL lastet opp i en frame.



Figur 5.2: Siden åpnet i live server, viser BL lastet opp på siden og fungerer slik som den andre.

Sårbarhet 10 – SENSITIV FIL TILGJENGELIG PÅ SIDEN

RISIKO: Medium

Beskrivelse

Filen phpinfo som inneholder sensitiv informasjon, er tilgjengelig på siden. Filen inneholder informasjon om systemet, php konfigurasjon, versjon, innstillinger, funksjonalitet, logg, hash-algoritmer og annen sensitiv informasjon.

Oppdagelse

Owasp zap scan varslet om at en hemmelig fil var funnet. Sjekket dette ved å kopiere og lime inn URL-en i en nettleser og fikk opp filen.

Konsekvenser

Denne filen forteller mye om serveren, hvilke systemer som brukes og hvordan de er satt opp. Dette gjør det veldig enkelt for en angriper å finne sårbarheter ved serveren, fordi dersom han finner filen, kan han bare søke opp sårbarheter ved systemene og deres versjoner. Siden serveren bruker mange utdaterte systemer.

Løsning

Flytt filen vekk fra web directory, slik at den ikke er tilgjengelig på nettet men lokalt på serveren.

Dokumentasjon

Ikke sikker

https://192.168.23.141/phpinfo.php

Mitt Kristiania

Kristiania

timeplan

Canvas

W3Schools

ChatGPT

OSWAP testing guide

Gmail

Google Disk - skb

Printer hk

Codecademy

PHP Version 7.3.31-1~deb10u5	
System	Linux osboxes 4.19.0-5-amd64 #1 SMP Debian 4.19.37-5 (2019-06-19) x86_64
Build Date	Sep 4 2023 21:49:25
Server API	CGI/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.3/cgi
Loaded Configuration File	/etc/php/7.3/cgi/php.ini
Scan this dir for additional .ini files	/etc/php/7.3/cgi/conf.d
Additional .ini files parsed	/etc/php/7.3/cgi/conf.d/10-mysqld.ini, /etc/php/7.3/cgi/conf.d/10-opcache.ini, /etc/php/7.3/cgi/conf.d/10-pdo.ini, /etc/php/7.3/cgi/conf.d/20-calendar.ini, /etc/php/7.3/cgi/conf.d/20-ctype.ini, /etc/php/7.3/cgi/conf.d/20-exif.ini, /etc/php/7.3/cgi/conf.d/20-fileinfo.ini, /etc/php/7.3/cgi/conf.d/20-ftp.ini, /etc/php/7.3/cgi/conf.d/20-gettext.ini, /etc/php/7.3/cgi/conf.d/20-iconv.ini, /etc/php/7.3/cgi/conf.d/20-json.ini, /etc/php/7.3/cgi/conf.d/20-mysqli.ini, /etc/php/7.3/cgi/conf.d/20-pdo_mysql.ini, /etc/php/7.3/cgi/conf.d/20-phar.ini, /etc/php/7.3/cgi/conf.d/20-posix.ini, /etc/php/7.3/cgi/conf.d/20-readline.ini, /etc/php/7.3/cgi/conf.d/20-shmop.ini, /etc/php/7.3/cgi/conf.d/20-sockets.ini, /etc/php/7.3/cgi/conf.d/20-sysvmsg.ini, /etc/php/7.3/cgi/conf.d/20-sysvsem.ini, /etc/php/7.3/cgi/conf.d/20-sysvshm.ini, /etc/php/7.3/cgi/conf.d/20-tokenizer.ini
PHP API	20180731
PHP Extension	20180731
Zend Extension	320180731
Zend Extension Build	API320180731.NTS
PHP Extension Build	API20180731.NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled
Zend Multibyte Support	disabled
IPv6 Support	enabled
DTrace Support	available, disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2
Registered Stream Filters	zlib.*, string.rot13, string.toupper, string.tolower, string.strip_tags, convert.*, consumed, dechunk, convert.iconv.*

Figur 3.1: en del av filen som er tilgjengelig

Sårbarhet 11 – DNS BIND VERSJON 9.11 UTDATERT

RISIKO: Medium

Beskrivelse

DNS er en protokoll som konverterer domene navn om til ip adresser og motsatt.

Serveren bruker BIND 9.11.5-P4-5.1+deb10u9-Debian som er en litt eldre versjon av BIND med noen kjente sårbarheter.

Oppdagelse

Nmap scan av åpne porter på serveren

Sårbarheter

- CVE-2023-50387
- CVE-2023-4408
- CVE-2023-50868
- CVE-2023-3341
- CVE-2022-3094
- CVE-2021-25216
- CVE-2021-25215

(spoofing, DNS cache poisoning, DNS tunneling, sub domain brute forcing)

Konsekvenser

Noen av konsekvensene av å ha en eldre versjon av BIND vil være utnyttning av de kjente sårbarhetene. Her kan en angriper utføre spoofing, DNS cache poisoning, DNS tunneling osv. Siden ip-adressen ennå ikke har fått tildelt et domenenavn, vil det være vanskelig å utnytte disse sårbarhetene på denne siden. Her er det ikke en veldig stor reel risiko enda.

Løsning

Lukk porten inntil dere setter opp et domenenavn til siden. Oppdater til en nyere versjon av DNS BIND sånn som BIND 9.18.

Sårbarhet 12 – SQL ERROR VISES PÅ SIDEN

RISIKO: Medium

Beskrivelse

Redirect til en side med SQL syntaks feilmelding. Dette er sensitiv informasjon som ikke burde deles. I tillegg forteller siden noe om databasen som navn.

Oppdagelse

Owasp zap scan som reagerte på det + SQLi testing som sendte meg feilmeldingene fra databasen når syntaksen ikke stemte.

Konsekvenser

Gjør det lettere for en angriper å forstå seg på systemet og sparer han masse tid på å kartlegge. Hacking av nettsiden vil bli mye enklere og ta mye mindre tid.

Løsning

Fjern redirect til siden som viser feilmelding og fjern siden som viser feilmelding

Dokumentasjon



Figur 7.1: Side som redirectes når man prøver seg på enkelte angrep, som forteller noe om databasen og SQL syntaks.

Sårbarhet 13 – X-CONTENT-TYPE-OPTIONS HEADER MANGLER

RISIKO: Lav

Beskrivelse

Manglende X-Content-Type-Options header. Dette er en header som beskytter siden mot MIME-sniffing ved å si at innholdet skal være den MIME typen som er spesifisert i content-type forblir det den er satt til og ikke endres. Denne headeren angir at MIME-typene er bevisst konfigurert. MIME-typer hjelper servere med å fortelle nettlesere hva slags data en fil inneholder og hvordan de skal behandle de. Dette kan være fks html dokumenter, jpeg bilder og JSON data.

MIME-sniffing er en prosess hvor nettleseren ignorerer MIME-typen i content-type for en fil og prøver å gjette hva den inneholder basert på innholdet i seg selv. Det er ikke alltid at nettleseren gjetter riktig og dette kan føre til at brukeren er sårbar for flere typer angrep.

X-Content-Type-Options forteller nettleseren at den alltid skal følge MIME-typen spesifisert i Content-typen og forhindrer MIME-sniffing.

Oppdagelse

Denne sårbarheten ble oppdaget i nikto scan og owasp zap.

Konsekvenser

MIME-sniffing kan føre til XSS angrep hvor filer som ikke er kode, tolkes som kode, slik at en angriper kan injisere ondsinnet kode i en brukers nettleser.

Løsning

Sett en X-Content-Type-Options header til nosniff i koden.

Sårbarhet 14 – COOKIES WITHOUT HTTPONLY FLAG

RISIKO: Lav

Beskrivelse

Cookie uten httpOnly flag. HttpOnly flagget gjør at cookies ikke kan endres med javascript ved at de bare er tilgjengelige som http-forespørsler til serveren.

Oppdagelse

Denne sårbarheten ble oppdaget gjennom en owasp zap scan.

Konsekvenser

Konsekvensene av denne sårbarheten kan være XSS angrep. Cookiene kan bli tilgjengelige for en angriper gjennom injisering av javascript kode, slik at angriper kan lese cookiene til en bruker.

Dette kan igjen føre til session hijacking som er at en angriper får tilgang til sesjons ID slik at han kan logge seg inn på brukeren.

Angriper kan også manipulere cookiene slik at han kan få tilgang til større privilegier eller endre data i cookies.

Løsning

Løsningen vil være å legge til en httpOnly flag til cookiene, i koden.

Sårbarhet 15 – PORT 79 FINGER ÅPEN

RISIKO: Lav

Beskrivelse

Eldre protokoll som ble brukt til å hente informasjon om brukere på en server fra 1970 for å finne informasjon om brukere i et nettverk, hvem som er logget inn, aktivitet osv. Denne porten er utdatert og har noen kjente sårbarheter som at den er sårbar for DOS angrep, ormer og trojaner.

Oppdagelse

Nmap scan av åpne porter

Konsekvenser

Siden dette er en utdatert protokoll som ikke brukes lengre, er den heller ikke oppdatert på sikkerhetsrisikoer, så den vil være veldig sårbar for utnyttelse. Siden responsen er så stor, vil en angriper kunne utføre DOS angrep. Den er også sårbar for ormer og trojaner, så en angriper kan infisere server med virus.

Løsning

Lukke denne porten

Sårbarhet 16 – PORT 37 TIME ÅPEN

RISIKO: Lav

beskrivelse

Nettverksprotokoll, returnerer dato og tidspunkt, brukt for å synkronisere tid mellom maskiner. Gammel og utdatert fra 1980 tallet med flere kjente sårbarheter som DOS angrep, ormer og spoofing. Den er ikke veldig presis og har ingen måte å håndtere forsinkelser på.

Oppdagelse

Nmap scan av åpne porter

Konsekvenser

Siden port 37 time protokoller er så utdatert og gammel som den er, har den veldig lite sikkerhetstiltak. Dette gjør den veldig sårbar for moderne angrep slik som DOS, angrep, ormer som sober og adm, spoofing.

Løsning

Bytt til NTP og lukk port 37

Sårbarhet 17 – HSTS HEADER MANGLER

RISIKO: Lav

Beskrivelse

HSTS passer på at nettleseren alltid bruker https og ikke http. Uten denne headeren kan siden prøve å kobles til via http i stedet, selv om tls/ssl er konfigurert.

Oppdagelse

Owasp zap

Konsekvenser

Angriper kan prøve å få siden til å kobles via http i stede og utføre man in the middle angrep.

Løsning

Legge til en HSTS header til serveren i koden.

Sårbarhet 18 – COOKIES WITHOUT SECURE FLAG

RISIKO: Lav

Beskrivelse

Cookies uten secure flag. Secure flag passer på at cookies ikke er tilgjengelig for uautorisert tilgang. Den passer på at cookies ikke sendes ukryptert. Den forhindrer fks MITM angrep.

Oppdagelse

Owasp zap scan

Konsekvenser

MITM angrep hvor angriper kan få tilgang til sensitiv informasjon.

Løsning

Pass på at cookies blir sendt med secure flag.

Sårbarhet 19 – TLS KONFIG - RENEGOTIATION TILLATER DOS ANGREP

RISIKO: Lav

Beskrivelse

Siden renegotiation i TLS er konfigurert til å tillate client-initiated renegotiation kan en angriper sende flere forespørslser om ny renegotiation på kort tid for å utføre et DOS-angrep. Dette er fordi renegotiation tar mye plass og har stor respons, som da vil overbelaste serveren ved mange forespørslser.

Oppdagelse

Sslyze scan av ip adressen.

Konsekvenser

Hvis serveren blir overbelastet vil det bli vanskelig for andre brukere å bruke nettsiden og serveren kan krasje, noe som vil føre til at det ikke vil være mulig å bruke siden før den er oppe igjen.

Løsning

Deaktiver client-initiated renegotiation.

Sårbarhet 20 – SERVER LEAKS VERSION INFORMATION VIA SERVER http RESPONSE HEADER

RISIKO: Lav

Beskrivelse

Serveren forteller om versjonsnummer via http response header. Dette kan gjøre det enklere for en angriper å finne sårbarheter ved serveren og utnytte disse. Det er god praksis å skjule dette, selv om det ikke er nødvendig.

Oppdagelse

Nessus, owasp zap respons

HTTP/1.1 304 Not Modified

Date: Tue, 12 Nov 2024 12:11:40 GMT

Server: Abyss/2.16.9.1-X1-Linux AbyssLib/2.16.9.1

Konsekvenser

Gjør det enklere for en angriper å finne sårbarheter ved versjonen som kjører.

Løsning

Skjule versjonsnummer i http response header.

Sårbarhet 21 – PORT 139 ÅPEN

RISIKO: Lav

Beskrivelse

Port 137 er en eldre versjon for fil og skriverdeling i windows. I dag brukes denne lite og er erstattet med port 445 som er tryggere.

Oppdagelse

Nmap scan av åpne porter.

Sårbarheter

- CVE-2019-0708 Bluekeep
- CVE-2017-0144 Eternalblue
- SMB Null session

Løsning

Burde fjernes, bruk heller 445.

Sårbarhet 22 – AUTOFYLL PASSORD TILLATT

RISIKO: LAV

Beskrivelse

Siden tillater passord autofyll, i login felt. Dette er ikke en direkte sårbarhet mot serveren men øker risikoen for XSS angrep og phishing.

Oppdagelse

Nessus

Konsekvenser

Siden nettsiden er sårbar for XSS angrep, kan en angriper sende javascript kode som sender autofyll funksjonen til seg selv, slik at personen kan logge seg inn på brukere.

Løsning

Legg til «autocomplete=off» i de feltene for innlogging.

Sårbarhet 23 – BASIC AUTHENTICATION

RISIKO: LAV

beskrivelse

Noen sider på web serveren bruker basic http authentication for pålogging. Basic authentication er en måte å autentisere en bruker på, men anbefales ikke lenger å brukes ettersom at den ikke fungerer like bra som andre alternativer.

Oppdagelse

Nessus

Konsekvenser

En angriper kan fange opp trafikken i klartekst og få tilgang til login og passord.

Løsning

Erstatt basic authentication med OAuth protokollen.

Informasjon 1 – Falsk positiv SSTI

Beskrivelse

SSTI er et angrep hvor angriper skriver template kode inn i input felt, som deretter kjøres på serveren.

Oppdagelse

Owasp zap

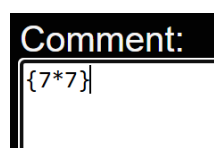
Gjennomgang av kildekode

Konklusjon

Konkluderer dette som en falsk sårbarhet etter grundig testing. Jeg oppdaget denne sårbarheten først ved en owasp zap scan som sa at input-feltene i guestbook.php var sårbare for SSTI. Siden owasp zap makerte denne sårbarheten som høy, valgte jeg å manuelt teste siden ved å skrive template kode som `{7*7}` og andre varianter inn i kommentar input feltene i guestbook.php. Da jeg gjorde dette, ble kommentaren postet uten tegnene. Dette antydte til at serveren saniterte inputet riktig ved å fjerne tegn som kunne tolkes som kode. Jeg testet i tillegg samme kode som owasp zap hadde brukt (`zj{{print "8424" "2247"}}zj`). Outputen viste ingen antydning til SSTI og det så ut som at siden filtrerte ut tegn som kan tolkes som kode.

I tillegg brukte jeg tplmap som ikke ga resultater på SSTI. I gjennomgang av kildekode, hvor jeg så på guestbook.php og security.php fant jeg ut at alt av input ble riktig sanitert slik at ssti ikke er mulig.

Dokumentasjon



Figur 1.1: Koden jeg skrev inn for å teste etter SSTI.



Figur 1.2: Kommentaren, hvor tegnene er fjernet.

```
if(isset($_POST['submit'])) {  
    $name = $_POST['name'];  
    $comment = $_POST['comment'];  
  
    $file = fopen('backend/comments.data', 'a');  
    fwrite($file, '<b>' . _sanitizeinput($name) . ' :</b> ' . _sanitizeinput($comment) . '<br>');  
    fclose($file);  
}
```

Figur 1.3: kode på *guestbook.php* som viser at name og comment blir sanitert.

```
<?php  
  
function _sanitizeinput($inputstring){  
    $result = preg_replace('/[^A-Za-z0-9,.\-!]/', '', $inputstring);  
    return $result;  
}  
  
?>
```

Figur 1.4: *_sanitizeinput* koden fjerner alt bortsett fra store og små bokstaver, tall og vanlige tegn som utroptegn osv. Dette vil si at kode som {} o.l. fjernes.

Informasjon 2 - FALSK POSITIV IP-ADRESSE I SERVER HEADER

Beskrivelse

Nikto varslet om at IP-adressen ble funnet i server headeren, dette var ikke en ip-adresse, men et versjonsnummer som nikto tolket som en ip-adresse pga punktumene i mellom tallene.

Oppdagelse

Nikto

Informasjon 3 – PORT 445 MICROSOFT-DS ÅPEN

Beskrivelse

Brukes for fil og skriverdeling i windows.

Oppdagelse

Nmap

Løsning

Sørg for at den bruker SMB2 eller 3

Informasjon 4 – PORT 9999 ABYSS ÅPEN

Beskrivelse

Abyss bruker port 9999, det finnes sårbarheter ved port 9999 så vær forsiktig med hvor mye denne brukes.

Informasjon 5 – PORT 9 ÅPEN

Det anbefales å lukke port 9, da den ikke har noe funksjon for serveren.

Informasjon 6 – PORT 13 ÅPEN

Denne porten returnerer dato og klokkeslett. Dette er en veldig gammel protokoll som vanligvis ble brukt for testing av nettverks tilkoblinger. I dag er denne protokollen ofte byttet ut med NTP port 123. anbefaler å lukke denne porten.